

CSEN 266 Artificial Intelligence
Group Homework #1 — Search
Fall 2026

Due: Wednesday, October 14, 11:59 p.m. (submit on Camino)

This is a GROUP assignment. Download `search_and_games.zip` from Camino and unzip it. You edit one file, `search.py`, where your search algorithms go.

Submission: one .zip file of your `search_and_games` folder, with `search.py` modified by you, submitted to Camino as a group. We will run the Experiments in Problem 1 and Problem 2 on your code, so make sure it works.

The following items are essential components of your algorithm. Explain them as tagged comments inside `breadthFirstSearch` and `aStarSearch`, directly in `search.py`. Write each one as **# TAG: your answer**, directly above the lines it describes. An answer may continue on the comment lines that follow it. Keep each tag spelled as shown, because the tags are checked when grading.

Tag	Question it answers	Where
G/H/F	How are the g, h, and f values obtained for a node?	A* only
GOAL TEST	Where do you call the goal test (<code>isGoalState</code>)?	A* only
FRONTIER	How is the frontier defined?	BFS and A*
VISITED SET	How are repeated states recorded and skipped?	BFS and A*
SUCCESSORS	How are the children of node generated, and how do they enter the frontier?	BFS and A*
PATH	How is the solution path returned?	BFS and A*

Both functions call the goal test the same way: `problem.isGoalState(state)` returns `True` or `False`, and `state` here is `node.state`. Only `aStarSearch` needs a GOAL TEST tag, so it is not shown in Figure 2: find the line where you call `isGoalState` in your own code, and write your GOAL TEST tag directly above it.

Figures 1 and 2 give each function as a skeleton. Each tag line holds the question from the table, and the CALLS block under it lists the functions, constructors and fields that stage uses. A name in quotes, such as `util.Queue()` “frontier”, is the variable the later calls in that block are written against. Working out the order and the control flow is your job. Both functions return a list of actions leading from the start to the goal, such as `['South', 'West']`, or an empty list when there is no path. Write your code below each block, then replace each question with your own answer, which describes your own code. You may keep or delete the CALLS lines.

Grading uses the tagged comments and the Experiments. You do not need `autograder.py` for this assignment. It checks eight questions, and this handout assigns two of them. Its question 4 can also fail on a correct A*.

Academic Dishonesty: We will be checking your code against other groups’ submissions in the class for logical redundancy. If you copy someone else’s code and submit it with minor changes, we will know, and we will pursue the strongest consequences available to us.

```

search.py
def breadthFirstSearch(problem: SearchProblem) -> List[Directions]:
    # FRONTIER: How is the frontier defined?
    # CALLS: util.Queue() "frontier", frontier.push(item), frontier.pop(),
    #         frontier.isEmpty(), problem.getStartState(),
    #         Node(state, parent, action, path_cost)

    # VISITED SET: How are repeated states recorded and skipped?
    # CALLS: a Python set "visited", visited.add(state), node.state,
    #         frontier.pop(), frontier.isEmpty()

    # SUCCESSORS: How are the children of node generated, and how do
    # they enter the frontier?
    # CALLS: problem.getSuccessors(state) -> [(next_state, action, step_cost), ...],
    #         Node(state, parent, action, path_cost), node.path_cost,
    #         frontier.push(item)

    # PATH: How is the solution path returned?
    # CALLS: a Python list "actions", node.parent, node.action, actions.insert(0, action)

```

Figure 1: The breadthFirstSearch skeleton. Write your code below each CALLS block.

```

search.py
def aStarSearch(problem: SearchProblem, heuristic=nullHeuristic) -> List[Directions]:
    # G/H/F: How are the g, h, and f values obtained for a node?
    # CALLS: node.path_cost (g), heuristic(state, problem) (h)

    # FRONTIER: How is the frontier defined?
    # CALLS: util.PriorityQueue() "frontier", frontier.push(item, priority),
    #         frontier.pop(),
    #         frontier.isEmpty(), problem.getStartState(),
    #         Node(state, parent, action, path_cost), heuristic(state, problem)

    # VISITED SET: How are repeated states recorded and skipped?
    # CALLS: a Python set or dictionary "visited", visited.add(state) or
    #         visited[state], node.state, node.path_cost,
    #         frontier.pop(), frontier.isEmpty()

    # SUCCESSORS: How are the children of node generated, and how do
    # they enter the frontier?
    # CALLS: problem.getSuccessors(state) -> [(next_state, action, step_cost), ...],
    #         Node(state, parent, action, path_cost), node.path_cost,
    #         heuristic(state, problem), frontier.push(item, priority)

    # PATH: How is the solution path returned?
    # CALLS: a Python list "actions", node.parent, node.action, actions.insert(0, action)

```

Figure 2: The aStarSearch skeleton. It follows Figure 1, with path costs and a heuristic added.

This assignment is adapted from the Pacman AI projects developed at UC Berkeley, <http://ai.berkeley.edu>.

Before you start

GW0 walked through installing Python and running `python pacman.py`. If you have not done GW0, do it first. Two things it does not cover:

```
python pacman.py -h
```

lists every option. Each option has a long form (`--layout`) and a short one (`-l`). And every command in this handout is also in `commands.txt`, for copying and pasting. If Pacman gets stuck, exit the game by typing CTRL-c into your terminal.

Problem 1 (Breadth-First Search) In the `breadthFirstSearch` function in `search.py`, implement breadth-first search with a visited set (a node whose state has already been expanded is skipped), for Pacman to find the path to the dot in the maze. Figure 1 is the skeleton for this function.

Experiments: Test your code using:

```
python pacman.py -l tinyMaze -p SearchAgent -a fn=breadthFirstSearch
python pacman.py -l smallMaze -p SearchAgent -a fn=breadthFirstSearch
python pacman.py -l mediumMaze -p SearchAgent -a fn=breadthFirstSearch -z .5 --frameTime 0
python pacman.py -l bigMaze -p SearchAgent -a fn=breadthFirstSearch -z .5 --frameTime 0
```

Problem 2 (A* Search) Implement A* search with a visited set in the empty function `aStarSearch` in `search.py`, for Pacman to find the path to the dot in the maze. Figure 2 is the skeleton for this function. The Experiments below pass `manhattanHeuristic` (the Manhattan distance to the goal, in `searchAgents.py`) in as the `heuristic` parameter.

Experiments: Test your code using:

```
python pacman.py -l tinyMaze -p SearchAgent -a fn=astar,heuristic=manhattanHeuristic
python pacman.py -l smallMaze -p SearchAgent -a fn=astar,heuristic=manhattanHeuristic
python pacman.py -l mediumMaze -z .5 -p SearchAgent -a fn=astar,heuristic=manhattanHeuristic --frameTime 0
python pacman.py -l bigMaze -z .5 -p SearchAgent -a fn=astar,heuristic=manhattanHeuristic --frameTime 0
```

Expected output. Each Experiment prints the cost of the path found and the number of search nodes expanded. For `tinyMaze` with BFS, the terminal shows:

```
[SearchAgent] using function breadthFirstSearch
[SearchAgent] using problem type PositionSearchProblem
Path found with total cost of 8 in 0.0 seconds
Search nodes expanded: 15
Pacman emerges victorious! Score: 502
```

A correct implementation gives these results for the eight Experiments:

Maze	BFS		A* (manhattanHeuristic)	
	Path cost	Nodes expanded	Path cost	Nodes expanded
tinyMaze	8	15	8	14
smallMaze	19	92	19	53
mediumMaze	68	269	68	221
bigMaze	210	620	210	549

Your path costs must match. Your node counts may differ slightly, because the order in which ties are broken and the point where the goal test is called both change the count.

If Pacman moves too slowly for you, try the option `--frameTime 0`.